

Flume: A Lightweight Framework for Engineering Design Optimization based on Directed Acyclic Graphs

Cameron S. Smith¹ and Graeme J. Kennedy¹

¹ Georgia Institute of Technology, United States  Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: 

Submitted: 11 March 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/))

Summary

Engineering design problems often involve a sequence of coupled analyses that are used to compute quantities of interest within an optimization problem. Derivative-based optimization methods provide a scalable way to solve these design problems if the derivatives can be computed efficiently. To achieve scalability and efficiency, the adjoint method is used to calculate the derivatives of the quantities of interest from the analysis sequence that constitute a system.

To more effectively organize and solve these design problems, we have developed a framework to facilitate the system-level construction and execution for both the forward analysis and adjoint-based derivative evaluation. This framework, entitled *Flume*, is designed around systems that can be described by a directed acyclic graph (DAG). While this specific architecture excludes systems that have implicitly coupled relationships, it is applicable for a variety of problems. Following the DAG structure, each node represents an individual analysis that needs to be performed for the optimization problem, and the edges represent connections between analyses and denote the flow of information. By describing a system in this manner, the DAG structure can be constructed programmatically, motivating the development of a framework to address this task.

To ensure that the framework is extensible, lightweight, and minimalistic, three primary classes have been implemented in Python to capture the necessary functionality: *State*, *Analysis*, and *System*. The first class, *State*, provides an object that stores numerical data along with some additional metadata, including type, shape, and object source information. *Analysis* is the foundation of *Flume*, and its primary tasks are to execute the forward and adjoint procedures to obtain the outputs and derivatives needed for optimization. Finally, *System* provides a set of methods to declare the objective function, constraints, and design variables that will be used within the optimization problem, as well as a means to visualize the DAG. To utilize *Flume*, a user's primary responsibility is to construct the individual analyses, inherited from the *Analysis* base class, that are needed for their *System*. By adhering to a set of naming and architectural requirements when scripting these analyses, the framework's backend will automatically connect outputs to variables that share the same name. This provides the user with a streamlined workflow, enabling them to focus on implementing new features and procedures instead of managing the integration.

The structure of *Flume* is visualized in [Figure 1](#), which depicts an abstracted *System* that encapsulates four distinct *Analysis* objects. Arrows that link *Analysis* objects denote *State* objects that connect outputs of one discipline to variables another. *Analysis* objects that are outlined in red and are labeled with "Top-level *Analysis* Object" are those that define output *States* that are utilized for optimization. Thus, the arrows that extend beyond the *System* boundary are *States* that define design variables, the objective function, or constraint functions for an optimization problem that is wrapped within the framework.

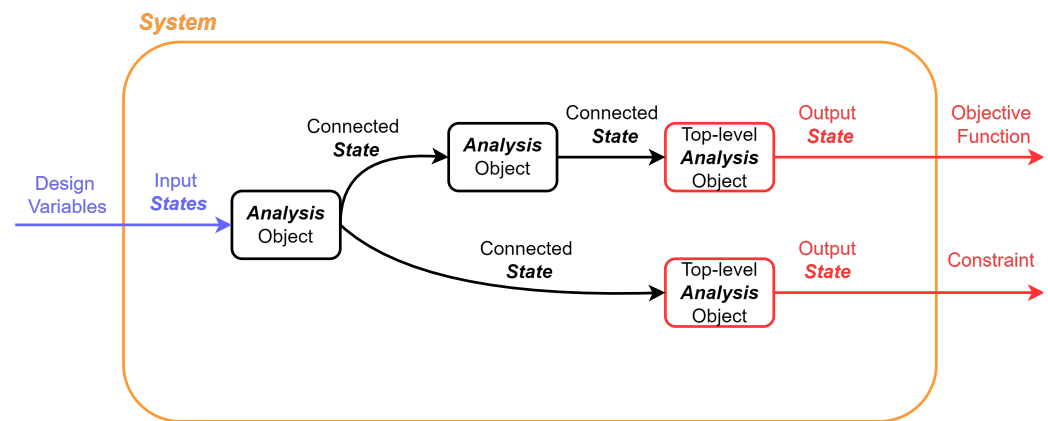


Figure 1: Abstracted System that illustrates the structure of the framework. Here, *State*, *Analysis*, and *System* are emphasized to denote the use of the primary classes provided within the library. Arrows extending beyond the boundary of the *System* denote quantities that are utilized for numerical optimization.

Statement of need

Frameworks for organizing and solving engineering design optimization problems have a common set of features and requirements, including modularity, intuitive user interfaces, object-oriented principles, and minimal overhead (Padula & Gillian, 2006; Salas & Townsend, 1998). For optimization problems with many design variables and few constraint functions, the adjoint method becomes the most computationally efficient and scalable way to compute the required derivatives (Martins & Ning, 2022). Several commercial software tools are available for solving engineering optimization problems, such as the *Adaptive Modeling Language suite* (Adaptive Modeling Language, 2026), *HEEDS* (Simcenter HEEDS, 2026), *ISight* (Isight, 2026), *ModelCenter* (ModelCenter, 2026), and *modeFRONTIER* (modeFRONTIER, 2026), but they are not open-source. Some open-source frameworks, such as *MACH* (Kenway et al., 2014) and *FUNtoFEM* (Jacobson et al., 2018), are successful in solving coupled aerostructural design optimization problems, but their use for this specific type of problem makes their extension to other disciplines more challenging. *pyOptSparse* (Wu et al., 2020) is a more general framework for solving constrained nonlinear optimization problems. However, *pyOptSparse*'s object-oriented approach does not easily define a hierarchical approach for constructing analysis sequences, which introduces implementation complexity when the adjoint method is required. *OpenMDAO* (Gray et al., 2019) addresses this organizational challenge by utilizing a hierarchical approach, and *MPhys* (Yildirim et al., 2025) provides functionality to integrate simulation software within the *OpenMDAO* framework. Derivative evaluation is facilitated using the modular analysis and unified derivatives (MAUD) architecture (Hwang & Martins, 2018), but the *OpenMDAO* framework has many requirements and base classes that create implementation challenges if extending its functionality.

While it is evident that users are presented with several viable options to perform numerical optimization, *Flume* was created to address the challenges identified above with the existing frameworks. First, *Flume* is particularly architected to facilitate the assembly of total derivatives using the adjoint method to perform gradient-based optimization. This ensures that the framework can be applied to a variety of problems with many design variables and few constraints while preserving scalability and computational efficiency. Object-oriented principles and inheritance are used, but the framework is also minimalistic with only three primary classes to ensure it is easily extensible. A hierarchical approach also facilitates the derivative evaluation, as each individual analysis discipline must only consider its respective variable-output combination. Then, based on the adjoint method, the framework is responsible for

propagating information through the model to compute the total derivative information that is used for optimization. This provides users with a streamlined approach, where they are primarily responsible for implementing the forward analysis and derivative evaluation techniques, while *Flume* orchestrates the organization and execution of the model.

Software design

During the construction of the *Flume* framework, several design decisions were made to prioritize flexibility and simplicity. The framework is written entirely in Python, and the only core numerical requirement is NumPy to support array operations. Within the framework, an object-oriented approach is employed with three primary classes:

- *State*: an object that defines the numerical value and its derivative for variables and outputs in the framework. It also stores additional metadata, such as its type, shape, description, and object source information.
- *Analysis*: an abstract base class that users inherit from to define two member functions. The first function uses variables, instances of *State*, to compute outputs, which are also *State* instances. The second function calculates the derivatives using the adjoint method, propagating derivatives from the output instances to accumulate contributions to the input instances.
- *System*: a container for *Analysis* objects that defines an analysis sequence. Also, it provides a set of methods to declare the objective function, constraints, and design variables that characterize an optimization problem.

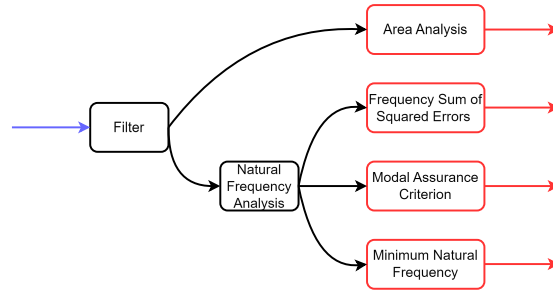
When defining analysis disciplines, users inherit from the *Analysis* base class, while *State* and *System* objects are instantiated directly. Relying on these three classes provides a minimal application programming interface (API), but their generality ensures that the framework is modular and can be supplemented with additional features for a user's specific needs. For example, C++ can be integrated with pybind11 to leverage the benefits of a compiled language for computationally-intensive analyses. This also enables the use of external tools that implement adjoint-based methods to compute derivatives for gradient-based optimization.

The core components of *Flume* enable the forward analysis and adjoint-based derivative evaluation. To perform optimization, two optimizer interfaces are currently supported with SciPy (Virtanen et al., 2020) and ParOpt (Chin et al., 2019). These interfaces provide the means of connecting an instance of *System* to an optimizer that will perform the numerical optimization. This highlights the practicality of the framework, as it contains the necessary functionality to quickly translate an analysis procedure into a design optimization problem.

Research impact statement

To date, the framework has been implemented for two applications beyond the simpler examples provided within the repository, which detail the construction of *Analysis* disciplines and *System* architectures. *Flume* has been used for topology optimization, with specific applications for initial post-buckling behavior (Smith et al., 2026) and inverse design problems. The network complexity of these systems, such as in Figure 2a and in Figure 3a, emphasizes the importance of a tool like *Flume*. The representative *System* diagrams and examples of the topology optimization formulations applied to a sample domain are given for the inverse design and post-buckling problems in Figure 2 and Figure 3, respectively.

(a) System defined within *Flume* that defines the optimization problem for inverse design applied to natural frequency problems



(b) Design domain in consideration for the topology optimization

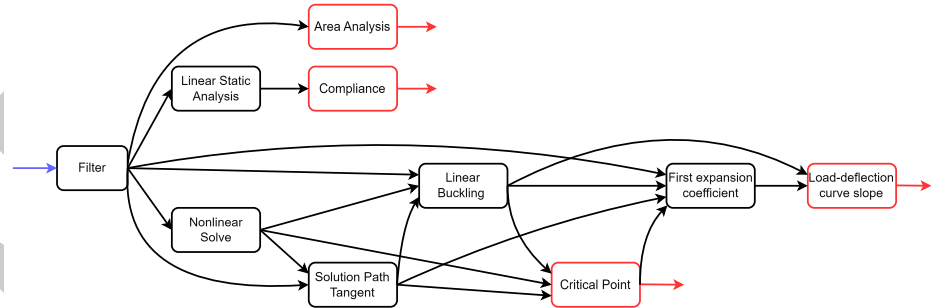


(c) Sample design from the topology optimization

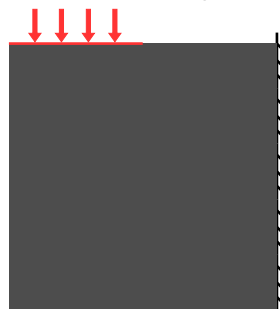


Figure 2: Demonstration of *Flume* applied to topology optimization for inverse design with natural frequency applications.

(a) System defined within *Flume* that defines the optimization problem for buckling load factor maximization with considerations for initial post-buckling behavior



(b) Design domain in consideration for the topology optimization with initial post-buckling consideration



(c) Sample design from the topology optimization

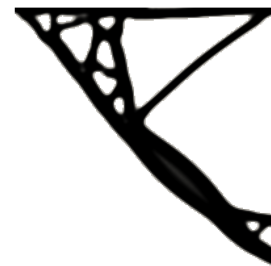


Figure 3: Demonstration of *Flume* applied to topology optimization for buckling load factor maximization with considerations for initial post-buckling behavior.

118 *Flume* by Example: Constrained Rosenbrock

119 To demonstrate the application of *Flume* and how a user interfaces with the *Analysis* base
 120 class and *System* class, a constrained Rosenbrock problem is considered in this section. The
 121 optimization statement for this example is given by

$$\begin{aligned} \min_{x,y} \quad & f(x,y) = (a-x)^2 + b(y-x^2)^2 \\ \text{s.t.} \quad & g(x,y) = x^2 + y^2 \leq 1 \end{aligned}$$

122 Within *Flume*, this is implemented by constructing three distinct *Analysis* objects: one to
 123 define the design variables, another to compute the objective function, and the last to compute
 124 the inequality constraint. As a demonstration, the code for the objective function *Analysis*
 125 class is included below. Here, it is worth discussing a few key features regarding the structure
 126 of the code.

- 127 ▪ The `__init__` method is responsible for defining the default parameter and variable
 128 values for the class. This specifies the full set of inputs that are required to compute
 129 the outputs, where, generally, parameters are fixed inputs and variables will nominally
 130 change throughout the process of an optimization. The variables and parameters are
 131 dictionaries that are stored as attributes of the associated class, and the variables are
 132 instances of the *State* class.
- 133 ▪ The `_analyze` method defines the forward analysis for evaluating the value of the
 134 Rosenbrock function. This utilizes the parameter and variable values stored within the
 135 class, and then the output *State* is assigned into an outputs dictionary.
- 136 ▪ The `_analyze_adjoint` method performs the adjoint analysis for the computations
 137 associated with the current class. Here, the variables are treated as independent, and
 138 the contributions from the adjoint variables associated with the outputs are accumulated
 139 into any previously computed derivatives for the variables.

140 When creating subclasses that inherit from *Analysis*, the underscore is used to indicate that
 141 `_analyze` and `_analyze_adjoint` are private, helper methods. This follows the *Template*
 142 *Method* behavioral design pattern (Gamma et al., 1994), where the user must provide the
 143 internal hooks to perform the overarching forward analysis and derivative evaluation procedures.

```
class Rosenbrock(Analysis):

    def __init__(self, obj_name: str, sub_analyses=[], **kwargs):

        # Set the default parameters
        self.default_parameters = {"a": 1.0, "b": 100.0}

        # Perform the base class object initialization
        super().__init__(obj_name=obj_name, sub_analyses=sub_analyses, **kwargs)

        # Set the default State for the variables
        xvar = State(value=0.0, desc="x state value", source=self)
        yvar = State(value=0.0, desc="y state value", source=self)

        # Construct variables dictionary
        self.variables = {"x": xvar, "y": yvar}

    return

    def _analyze(self):
        # Extract the variable values
        x = self.variables["x"].value
        y = self.variables["y"].value
```

```

# Extract the parameter values
a = self.parameters["a"]
b = self.parameters["b"]

# Compute the value of the Rosenbrock function
f = (a - x) ** 2 + b * (y - x**2) ** 2

# Store the outputs
self.outputs = {}

self.outputs["f"] = State(
    value=f, desc="Rosenbrock function value", source=self
)

return

def _analyze_adjoint(self):
    # Extract the derivatives of the outputs
    fb = self.outputs["f"].deriv

    # Extract the variable values
    x = self.variables["x"].value
    y = self.variables["y"].value

    # Extract the variable derivatives
    xb = self.variables["x"].deriv
    yb = self.variables["y"].deriv

    # Extract the parameter values
    a = self.parameters["a"]
    b = self.parameters["b"]

    # Compute xb
    xb += (2 * (a - x) * -1 + 2.0 * b * (y - x**2) * -2 * x) * fb

    # Compute yb
    yb += (2 * b * (y - x**2)) * fb

    # Assign the derivative values
    self.variables["x"].set_deriv_value(deriv_val=xb)
    self.variables["y"].set_deriv_value(deriv_val=yb)

    return

```

144 The *Analysis* classes that define the design variables and compute the constraint function are
 145 similar in structure to the one above. Next, the section below outlines how the user sets up a
 146 *System* and optimizes with the *FlumeScipyInterface*. Again, a few salient points are discussed.

- 147 ■ Instances for the *RosenbrockDVs*, *Rosenbrock* and *RosenbrockConstraint* objects are
 148 each constructed. Here, the instance for *RosenbrockDVs* is passed as a sub-analysis to
 149 the *Rosenbrock* and *RosenbrockConstraint* objects during construction, which establishes
 150 a connection between the *State* objects for *x* and *y*. This ensures that the same values
 151 are used when computing the objective or constraint function and provides paths that
 152 trace back to the same set of design variables.
- 153 ■ The *System* is constructed, where the top-level analyses are provided as a list. Effectively,
 154 this list defines the *Analysis* objects that are responsible for computing the objective
 155 and constraints for the optimization problem. Any sub-analyses are not required to
 156 be provided here, as this information is encoded within the object construction for the

- 157 top-level analyses.
- 158 ■ The design variables, objective, and constraints are all declared for the *System*, which
- 159 are stored and accessed when defining the optimization problem. Here, these quantities
- 160 are declared by using the global names, which are given by `obj_name.local_name`. The
- 161 user can also provide information for design variable bounds and constraint direction
- 162 and right-hand side values.
- 163 ■ Finally, in this example, the *FlumeScipyInterface* is utilized to formulate an optimization
- 164 problem using the design variable, objective, and constraints declared for the *System*.
- 165 This interface internally wraps SciPy optimize's `minimize` function, and the method and
- 166 options can be controlled by the user.

```
# Construct the design variables object
rosenbrock_dvs = RosenbrockDVs(obj_name="dvs", sub_analyses=[])

# Construct the analysis object for the Rosenbrock function
a = 1.0
b = 100.0

rosenbrock = Rosenbrock(
    obj_name="rosenbrock", sub_analyses=[rosenbrock_dvs], a=a, b=b
)

# Construct the analysis object for the constraint on the design variables
rosenbrock_con = RosenbrockConstraint(
    obj_name="con", sub_analyses=[rosenbrock_dvs]
)

# Construct the system
sys = System(
    sys_name="rosen_sys_con",
    top_level_analysis_list=[rosenbrock, rosenbrock_con],
    log_name="flume.log",
    log_prefix="tests/rosenbrock_constrained",
)

# Declare the design variables for the system
sys.declare_design_vars(
    global_var_name={
        "dvs.x_dv": {"lb": -1.5, "ub": 1.5},
        "dvs.y_dv": {"lb": -1.5, "ub": 1.5},
    }
)

# Declare the objective
sys.declare_objective(global_obj_name="rosenbrock.f")

# Declare the constraint
sys.declare_constraints(
    global_con_name={"con.g": {"direction": "leq", "rhs": 1.0}}
)

# Construct the Scipy interface
interface = FlumeScipyInterface(flume_sys=self.flume_sys)

# Set a random starting point
x0 = np.random.uniform(low=-5.0, high=5.0, size=2)

# Optimize the problem with SciPy minimize
x, res = interface.optimize_system(x0=x0, method="SLSQP")
```

167 AI usage disclosure

168 Regarding code development for *Flume*, generative AI tools, specifically ChatGPT, were
 169 minimally used to assist with interpreting Python error statements that were encountered
 170 during the framework's construction. Beyond this use case, generative AI was not utilized to
 171 write any software, sections of the manuscript, or the documentation.

172 *Adaptive modeling language*. (2026). [https://www.technosoft.com/application-](https://www.technosoft.com/application-software/adaptive-modeling-language/)
 173 [software/adaptive-modeling-language/](https://www.technosoft.com/application-software/adaptive-modeling-language/).

174 Chin, T. W., Leader, M. K., & Kennedy, G. J. (2019). A scalable framework for large-scale
 175 3D multimaterial topology optimization with octree-based mesh adaptation. *Advances in*
 176 *Engineering Software*, 135. <https://doi.org/10.1016/j.advengsoft.2019.05.004>

177 Gamma, E., Helm, R., Johnson, R., & Vlissides, J. M. (1994). *Design patterns: Elements of*
 178 *reusable object-oriented software* (1st ed.). Addison-Wesley Professional. ISBN: 0201633612

179 Gray, J. S., Hwang, J. T., Martins, J. R., Moore, K. T., & Naylor, B. A. (2019). OpenMDAO: An
 180 open-source framework for multidisciplinary design, analysis, and optimization. *Structural*
 181 *and Multidisciplinary Optimization*, 59(4), 1075–1104.

182 Hwang, J. T., & Martins, J. R. R. A. (2018). A computational architecture for coupling
 183 heterogeneous numerical models and computing coupled derivatives. *ACM Transactions on*
 184 *Mathematical Software*, 44(4), 1–39. <https://doi.org/10.1145/3182393>

185 *Isight*. (2026). <https://www.3ds.com/products/simulia/isight>.

186 Jacobson, K., Kiviaho, J. F., Smith, M. J., & Kennedy, G. (2018, January). An
 187 aeroelastic coupling framework for time-accurate analysis and optimization. *2018*
 188 *AIAA/ASCE/AHS/ASC Structures, Structural Dynamics, and Materials Conference*.
 189 <https://doi.org/10.2514/6.2018-0100>

190 Kenway, G. K. W., Kennedy, G. J., & Martins, J. R. R. A. (2014). Scalable parallel approach
 191 for high-fidelity steady-state aeroelastic analysis and adjoint derivative computations. *AIAA*
 192 *Journal*, 52(5), 935–951. <https://doi.org/10.2514/1.j052255>

193 Martins, J. R. R. A., & Ning, A. (2022). *Engineering design optimization*. Cambridge University
 194 Press. <https://doi.org/10.1017/9781108980647>

195 *modeFRONTIER*. (2026). <https://engineering.esteco.com/modelfrontier/>.

196 *ModelCenter*. (2026). <https://www.ansys.com/products/connect/ansys-modelcenter>.

197 Padula, S., & Gillian, R. (2006). Multidisciplinary environments: A history of engineering
 198 framework development. *11th AIAA/ISSMO Multidisciplinary Analysis and Optimization*
 199 *Conference*, 7083.

200 Salas, A., & Townsend, J. (1998). Framework requirements for MDO application development.
 201 *7th AIAA/USAF/NASA/ISSMO Symposium on Multidisciplinary Analysis and Optimization*,
 202 4740.

203 *Simcenter HEEDS*. (2026). [https://www.siemens.com/en-us/products/simcenter/integration-](https://www.siemens.com/en-us/products/simcenter/integration-solutions/heeds/)
 204 [solutions/heeds/](https://www.siemens.com/en-us/products/simcenter/integration-solutions/heeds/).

205 Smith, C., Li, Bao, & Kennedy, G. J. (2026). Topology optimization for initial post-
 206 buckling structural response: An application of koiter asymptotic theory. In *Structural and*
 207 *Multidisciplinary Optimization*. Springer.

208 Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D.,
 209 Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson,
 210 J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., ... SciPy
 211 1.0 Contributors. (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in
 212 Python. *Nature Methods*, 17, 261–272. <https://doi.org/10.1038/s41592-019-0686-2>

- 213 Wu, E., Kenway, G., Mader, C. A., Jasa, J., & Martins, J. R. R. A. (2020). pyOptSparse:
214 A python framework for large-scale constrained nonlinear optimization of sparse systems.
215 *Journal of Open Source Software*, 5(54), 2564. <https://doi.org/10.21105/joss.02564>
- 216 Yildirim, A., Jacobson, K. E., Anibal, J. L., Stanford, B. K., Gray, J. S., Mader, C. A.,
217 Martins, J. R. R. A., & Kennedy, G. J. (2025). MPhys: A modular multiphysics library for
218 coupled simulation and adjoint derivative computation. *Structural and Multidisciplinary*
219 *Optimization*, 68(1). <https://doi.org/10.1007/s00158-024-03900-0>

DRAFT